

Повторная загрузка PDF и социальная библиотека

Подробное исполняемое ТЗ для Qwen 3.8 Max Preview: от подтверждённой причины HTTP 500 до username, slug, профилей, лайков, комментариев и полного E2E-контроля.

Модель	qwen3.8-max-preview
Режим	Thinking всегда включён
Стратегия	Baseline -> отдельная ветка -> поэтапные коммиты
Дата	11 августа 2026

Навигация

Документ повторяет локальный handoff-файл и рассчитан на прямую передачу coding-агенту. Разделы расположены в порядке выполнения.

- 01 1. Роль и режим работы
- 02 2. Подтверждённая причина текущей ошибки
- 03 3. Целевой результат
- 04 4. Архитектура проекта, которую нужно сохранить
- 05 5. Этап A: исправление удаления и повторной загрузки
- 06 6. Этап B: username, профиль автора и slug
- 07 7. Этап C: социальные таблицы и API
- 08 8. Этап D: публичный Next.js-интерфейс
- 09 9. Обновление запросов без N+1
- 10 10. Полный набор проверок
- 11 11. Критерии приёмки
- 12 12. Как запустить это задание через Qwen Code
- 13 13. Источники конфигурации Qwen

Этот файл - исполняемое техническое задание для Qwen Code. Он намеренно не входит в Git: исходная точка уже должна быть сохранена отдельным baseline-коммитом, а результат работы агента должен быть виден как чистый diff поверх него.

1. Роль и режим работы

Ты работаешь как ведущий full-stack разработчик MyCharacter. Нужно исправить повторную загрузку удалённого PDF и превратить community-библиотеку шаблонов в небольшой социальный каталог: авторы с username, стабильные slug-ссылки, публичные профили, лайки и комментарии.

Работай автономно до выполнения критериев приёмки, но соблюдай ограничения:

- Не удаляй и не перезаписывай `.env.local`, пользовательские PDF и данные Docker.
- Никогда не запускай `docker compose down -v`.
- Не используй `git reset --hard`, `git checkout --` и другие команды,

уничтожающие незакоммиченные изменения.

- Сначала прочитай текущий код, миграции, тесты и `git status`.
- Сохрани уже реализованные исправления загрузчика, QwenCloud, CORS и

русификации полей PDF.

- Не публикуй ключи и не печатай содержимое `.env.local`.
- Не меняй модель Qwen в приложении: оставить `qwen3.8-max-preview`, `thinking`

включён. У Preview-модели `reasoning` отключать нельзя.

- Все пользовательские строки добавить одновременно в `ru.json` и `en.json`.
- После каждого этапа запускать относящиеся к нему тесты. Перед завершением

обязательно выполнить полный `docker compose run --rm check`.

- Делай небольшие осмысленные коммиты по этапам. Не изменяй baseline-коммит.

2. Подтверждённая причина текущей ошибки

Пользователь загрузил `DnD5e_character_sheet_RUS (1).pdf`, затем удалил шаблон D&D 5e из библиотеки и попытался загрузить тот же PDF снова. UI показывает `An unexpected error occurred`.

Проверенные факты:

- Удалённая строка остаётся в `pdf_templates` с заполненным `deleted_at`.
- `PdfUploadService` ищет только активный дубликат (`deleted_at is null`), не находит его и пытается вставить новую строку.
- Индекс `pdf_templates_private_hash_idx` уникален по (`owner_id`, `sha256`), но его условие учитывает только `visibility = 'private'` и НЕ учитывает `deleted_at is null`.
- PostgreSQL возвращает `unique violation 23505`; обработчика этого случая нет, поэтому Fastify превращает его в общий HTTP 500.
- Конкретная удалённая запись на момент диагностики:

52fd08c3-4808-405c-83a3-43e703ac15b0, SHA-256
6b6a824d350c5b235796313db3587ae205d9cf90ef11262a453edeb16626ab5e.

- API недостаточно логирует контекст нештатной ошибки загрузки.

Это не ошибка имени файла. Дубликат определяется по содержимому PDF (SHA-256).

3. Целевой результат

Обязательно

1. Повторная загрузка удалённого PDF больше не даёт 500.
2. Если физический PDF и каталог ещё существуют, повторная загрузка прозрачно восстанавливает старый шаблон и обновляет введенные пользователем метаданные.
3. У пользователя есть username; публичный автор отображается как @username.
4. У каждого шаблона есть человекочитаемый slug, не меняющийся при переименовании заголовка.
5. Community-карточка ведёт на публичную страницу шаблона и профиль автора.
6. Публичный шаблон можно лайкнуть/разлайкнуть; операция идемпотентна.
7. На публичном шаблоне можно читать комментарии, а авторизованный пользователь может добавить комментарий и удалить собственный.
8. Удалённые шаблоны видны владельцу в разделе «Недавно удалённые» и могут быть восстановлены в течение существующего 30-дневного срока.
9. Ошибки загрузки показываются на русском/английском осмысленно, без общего An unexpected error occurred. для ожидаемых конфликтов.
10. Все новые операции имеют тесты и не открывают приватные шаблоны.

Не входит в первый релиз

- Личные сообщения, подписки на людей, бесконечная лента и алгоритмические рекомендации.
- Вложения и Markdown/HTML в комментариях.
- Дизлайки, рейтинги с весами, репутация и платные функции.
- Отдельная административная панель модерации. Подготовить структуру так, чтобы модерацию можно было добавить позже.

4. Архитектура проекта, которую нужно сохранить

- apps/api: Fastify REST API, авторизация через cookie и request.actor.
- apps/web: Next.js App Router, серверные страницы получают данные через @/lib/api/server, интерактивные мутации выполняют клиентские компоненты.
- packages/contracts: общие Zod-схемы и TypeScript-типы.
- packages/database: Kysely-типы и последовательные миграции.
- packages/storage: физическое хранение PDF.

- `apps/worker`: каталогизация PDF и окончательная очистка корзины.
- Текущая community-библиотека находится по

`/dashboard/systems/community`.

Не добавляй второй ORM, Supabase или отдельную базу. Используй существующие Fastify, Kysely, Next.js, next-intl и дизайн-токены проекта.

5. Этап А: исправление удаления и повторной загрузки

A1. Миграция индекса

Создай следующую по времени миграцию в `packages/database/migrations`.

1. Удали `pdf_templates_private_hash_idx`.
2. Создай его заново как уникальный индекс (`owner_id`, `sha256`) с условием:
`visibility = 'private' AND deleted_at IS NULL`.
3. Обнови тест миграций: условие индекса должно явно содержать
`deleted_at IS NULL`.
4. В `down` восстанови прежнюю форму индекса только если это возможно без
конфликтующих данных; иначе документируй ограничение `rollback`.

A2. Восстановление при повторной загрузке

В `apps/api/src/modules/pdf/upload-service.ts` добавь ветку для удалённого дубликата владельца.

Рекомендуемый алгоритм:

1. Посчитать и проверить PDF как сейчас.
2. В одной транзакции заблокировать подходящую строку владельца через
`FOR UPDATE`.
3. Если активная строка существует - вернуть `existing`.
4. Если существует удалённая строка и её `object_files.state = 'ready'`, а файл
физически доступен:
 - очистить `deleted_at`;
 - обновить `title`, `game_system`, `allow_vision`, `is_public`, `updated_at`;
 - не уничтожать поля каталога и связи уже созданных персонажей;
 - вернуть новый результат `restored` с прежним `templateId`.
5. Если метаданные старой записи есть, но файл уже удалён/помечен `deleting`,
не восстанавливать битую запись; создать новый шаблон после безопасной очистки или выдать
контролируемую ошибку, выбранную на основании текущих FK.
6. Если параллельные запросы всё же сталкиваются с 23505, повторно найти
активную запись и вернуть её вместо HTTP 500.
7. При восстановлении публичность не должна автоматически возвращаться из
старой записи: использовать текущее значение формы.

Не ориентируйся на имя файла: единственным критерием идентичности остаётся SHA-256 содержимого.

A3. Корзина шаблонов

- Расширить `templateScopeSchema` значением `trash`.
- `TemplateService.list(..., 'trash')` возвращает только удалённые шаблоны

текущего владельца, не старше 30 дней.

- Создать `/dashboard/systems/trash` и добавить вкладку в

`SystemsSectionTabs`.

- Карточка показывает срок окончательного удаления и кнопку «Восстановить».
- Восстановление использует существующий

`POST /api/templates/:id/restore`.

- Если активная копия того же SHA уже появилась, восстановление старой записи

должно вернуть контролируемый `409 TEMPLATE_DUPLICATE_ACTIVE`, а UI должен предложить открыть активную копию.

A4. Ошибки и наблюдаемость

- Преобразовать ожидаемые конфликты PostgreSQL в `AppError` с конкретным кодом.
- В UI сопоставлять коды ошибок с локализованным текстом, но сохранять

`requestId` для диагностики.

- Добавить структурный `server log` при неожиданной ошибке загрузки: `requestId`, `actorId`, размер и SHA PDF, стадия операции. Не логировать PDF и ключи.

A5. Тесты исправления

Минимально:

- загрузить PDF, удалить шаблон, загрузить те же bytes снова - HTTP 200/201,

тот же шаблон восстановлен, 500 нет;

- имя файла отличается, bytes совпадают - восстановление работает;
- `title/gameSystem` меняются при восстановлении;
- существующие `characters` продолжают ссылаться на рабочий `template`;
- активный дубликат возвращает `existing`;
- два параллельных `upload` не создают две активные строки;
- пользователь В не восстанавливает шаблон пользователя А;
- старый/физически удалённый файл обрабатывается контролируемо.

Первый отдельный коммит агента:

`fix(pdf): restore deleted templates on duplicate upload`

6. Этап В: username, профиль автора и slug

В1. Схема данных

В той же миграции или в отдельной следующей миграции добавить:

profiles

- username text;
- bio text not null default '';
- уникальный регистронезависимый индекс по lower(username);
- CHECK: username в lowercase, длина 3-30, шаблон

[a-z0-9][a-z0-9_-]{2,29};

- CHECK: char_length(bio) <= 500.

Существующим профилям сделать детерминированный backfill. Предпочтительно нормализовать local-part email; при пустом или конфликтном результате использовать user-<первые 8 символов uuid> и уникальный числовой суффикс. После backfill сделать username NOT NULL.

pdf_templates

- slug text;
- backfill существующих шаблонов безопасным значением

template-<первые 8 символов uuid>;

- затем slug NOT NULL;
- уникальный индекс (owner_id, slug) для строк с ненулевым owner;
- CHECK: длина 1-80, lowercase URL-safe [a-z0-9]+(?:-[a-z0-9]+)*.

Обновить packages/database/src/types.ts.

В2. Генерация username

- При регистрации генерировать username из local-part email.
- Нормализация не должна сохранять email целиком и домен.
- Разрешать редактирование username через профиль с проверкой занятости.
- Ошибка занятости: USERNAME_TAKEN (409).
- Если username разрешено менять неоднократно, явно принять решение по старым

URL. Для MVP допустимо ограничить смену и предупредить в UI. Не делать тихое переназначение чужого username.

В3. Генерация slug

Добавь чистую тестируемую функцию slugifyTemplateTitle.

- Lowercase.
- Транслитерация базовой кириллицы.
- D&D 5e превращается примерно в d-d-5e.
- Повторные разделители схлопываются в -.
- Максимум 72 символа для основы.

- Пустой результат -> template.
- Коллизии автора получают -2, -3 и т.д.
- Slug задаётся при создании/восстановлении только если отсутствует.
- Изменение title не меняет slug автоматически, чтобы внешние ссылки не ломались.
- Защититься от гонки уникальным индексом и ограниченным retry.

B4. Контракты профиля

В packages/contracts добавить схемы:

- usernameSchema;
- publicAuthorSchema: id, username, displayName;
- publicProfileSchema: username, displayName, bio, joinedAt, publicTemplateCount, totalLikes;
- updateMyProfileRequestSchema: displayName, username, bio;
- расширить community TemplateSummary: slug, author, likeCount, commentCount, likedByMe (последнее зависит от сессии).

B5. API профилей

Добавить модуль apps/api/src/modules/profiles:

- GET /api/profiles/:username - публичный профиль и его активные, подтверждённые, публичные шаблоны;
- GET /api/profiles/me - приватные настройки текущего пользователя;
- PATCH /api/profiles/me - изменение displayName/username/bio;
- публичный endpoint не должен возвращать email, locale, isAdmin или userId, если он не нужен клиенту.

Второй отдельный коммит агента:

feat(profiles): add usernames and stable template slugs

7. Этап С: социальные таблицы и API

C1. template_likes

Поля:

- user_id uuid not null references users(id) on delete cascade;
- template_id uuid not null references pdf_templates(id) on delete cascade;
- created_at timestampz not null default now();
- primary key (user_id, template_id);
- индекс (template_id, created_at desc).

C2. template_comments

Поля:

- id uuid primary key default gen_random_uuid();
- template_id uuid not null references pdf_templates(id) on delete cascade;
- author_id uuid not null references users(id) on delete cascade;
- body text not null;
- created_at, updated_at;
- CHECK: char_length(btrim(body)) between 1 and 2000;
- индекс (template_id, created_at desc, id desc).

Комментарии хранить как plain text. React должен экранировать текст. Не разрешать HTML.

C3. Социальные endpoint-ы

Предлагаемый REST:

- GET /api/community/:username/:slug - публичная карточка/детали;
- PUT /api/templates/:id/like - поставить лайк, idempotent;
- DELETE /api/templates/:id/like - убрать лайк, idempotent;
- GET /api/templates/:id/comments?cursor=...&limit=20 - публичный список;
- POST /api/templates/:id/comments - авторизованный комментарий;
- DELETE /api/templates/:id/comments/:commentId - автор комментария или администратор.

Для всех действий target должен быть:

- deleted_at is null;
- visibility = 'private';
- is_public = true;
- catalog_approved_at is not null;
- catalog_status in ('ready', 'partial').

Если любое условие не выполнено, наружу возвращать 404, а не раскрывать существование приватного шаблона.

Ограничения:

- POST комментария: rate limit примерно 10/мин на пользователя;
- body trim 1-2000;
- пагинация cursor-based или стабильная (created_at, id);
- лайки считаются одним SQL-запросом, без N+1;
- likedByMe вычислять через optional request.actor, GET остаётся публичным;
- удаление/скрытие шаблона немедленно убирает его из публичной выдачи.

C4. Тесты API

- like дважды -> одна строка и неизменный count;

- unlike дважды -> 204 и count 0;
- unauthenticated like/comment -> 401;
- GET public details/comments работает без сессии;
- private/unapproved/deleted template -> 404;
- comment trim, empty и >2000;
- чужой comment удалить нельзя;
- автор/администратор удаляет допустимо;
- community list возвращает author, slug и правильные counts;
- запрос списка не выполняет отдельный запрос на каждую карточку.

Третий отдельный коммит агента:

feat(community): add template likes and comments

8. Этап D: публичный Next.js-интерфейс

D1. Маршруты

Создать Server Component страницы:

- /users/[username] - публичный профиль автора;
- /community/[username]/[slug] - публичная страница шаблона;
- /dashboard/profile - настройки собственного профиля;
- /dashboard/systems/trash - корзина шаблонов.

В Next.js 15/16 params типизировать как Promise и await params. Публичные страницы должны использовать generateMetadata. Для отсутствующих ресурсов вызывать notFound(), а не рендерить пустую страницу.

D2. Community-карточка

Обновить community-template-grid.tsx:

- заголовок ведёт на /community/{username}/{slug};
- @username - отдельная ссылка на /users/{username};
- показывать количество лайков и комментариев;
- кнопка лайка имеет aria-pressed, понятный accessible label и optimistic UI с rollback при ошибке;
- существующая кнопка добавления шаблона в свои системы сохраняется;
- клики по автору/лайку не должны случайно активировать всю карточку.

D3. Страница шаблона

Показывать:

- title, gameSystem, число страниц, статус проверки;
- автора и ссылку на профиль;
- likeCount/commentCount;

- кнопку добавить в свои системы;
- список комментариев;
- форму комментария для вошедшего пользователя;
- призыв войти для гостя;
- понятные empty/loading/error состояния.

После POST комментария обновить список без полного перезапуска приложения. Допустим `router.refresh()`; лучше локально вставить сериализованный объект, если это не усложняет код.

D4. Профиль

Публичная страница профиля:

- @username, displayName, bio, дата регистрации;
- количество публичных шаблонов и суммарных лайков;
- сетка публичных шаблонов автора;
- пустое состояние.

Настройки профиля:

- поля displayName, username, bio;
- live-подсказка итогового URL;
- локализованные сообщения validation/USERNAME_TAKEN/success.

D5. Визуальные требования

- Использовать существующие Button, Input, CSS-переменные и радиусы.
- Контентная ширина должна быть согласована с остальным dashboard и не создавать случайную узкую колонку на широком экране.
- Mobile: одна колонка; desktop: карточка шаблона + social sidebar/комментарии.
- Не добавлять тяжёлую UI-библиотеку.
- Все icon-only кнопки имеют aria-label.
- Даты форматируются по текущей locale без hydration mismatch.

Четвёртый отдельный коммит агента:

feat(web): add public authors and social template pages

9. Обновление запросов без N+1

При расширении community list не загружай лайки, комментарии и автора в цикле. Используй один запрос с join/subquery/CTE:

- join profiles по owner_id;
- агрегат likes по template_id;
- агрегат comments по template_id;
- optional left join текущего user like;
- coalesce(count, 0);
- сортировка сначала по approved/updated или по популярности только после

отдельного продуктового решения.

Добавь индексы, соответствующие фактическим WHERE и ORDER BY.

10. Полный набор проверок

Перед завершением выполнить:

```
git diff --check
docker compose run --rm check
```

Затем поднять приложение и выполнить ручной/E2E сценарий:

1. Войти пользователем А.
2. Загрузить интерактивный PDF и дождаться каталога.
3. Подтвердить и опубликовать шаблон.
4. Удалить шаблон.
5. Убедиться, что он появился в корзине и исчез из community.
6. Загрузить те же bytes под другим именем файла.
7. Убедиться, что шаблон восстановлен без HTTP 500 и открывается.
8. Проверить URL `/community/{username}/{slug}`.
9. Выйти: публичный профиль, шаблон и комментарии читаются.
10. Войти пользователем В: поставить лайк и комментарий.
11. Обновить страницу: count и комментарий сохраняются.
12. Пользователь А не может удалить комментарий В, если он не администратор; пользователь В может удалить свой.
13. Скрыть/удалить шаблон: публичный URL возвращает 404.
14. Старые персонажи пользователя А по восстановленному шаблону всё ещё открываются и экспортируются в PDF.

Сохранить краткий отчёт агента:

- список созданных миграций;
- список endpoint-ов и страниц;
- результаты тестов;
- известные ограничения;
- `git log --oneline <baseline>..HEAD`;
- `git diff --stat <baseline>..HEAD`.

11. Критерии приёмки

Работа не завершена, пока все пункты не выполнены:

- ☐ Повторная загрузка удалённого PDF не даёт 500.
- ☐ Восстановление не ломает существующих персонажей.
- ☐ Индекс дубликатов игнорирует удалённые строки.
- ☐ Есть UI-корзина и восстановление шаблонов.

- ☐ Все пользователи имеют уникальный username.
- ☐ Новые шаблоны получают стабильный slug.
- ☐ Community показывает автора и social counts.
- ☐ Публичный профиль доступен по username.
- ☐ Публичный шаблон доступен по username + slug.
- ☐ Like/unlike идемпотентны.
- ☐ Комментарии валидируются и защищены правами.
- ☐ Приватные/удалённые шаблоны не утекли в public API.
- ☐ RU и EN переводы полны.
- ☐ TypeScript, lint и все тесты проходят.
- ☐ Полный E2E сценарий проверен.
- ☐ Изменения разбиты на понятные коммиты поверх baseline.

12. Как запустить это задание через Qwen Code

Установка на macOS

Официальный рекомендуемый вариант:

```
bash -c "$(curl -fsSL https://qwen-code-assets.oss-cn-hangzhou.aliyuncs.com/installation/install-qwen.sh)" -s --source bailian
qwen --version
```

Альтернатива при уже установленном Node.js 22+:

```
npm install -g @qwen-code/qwen-code@latest
qwen --version
```

Настройка Token Plan Personal

В `~/qwen/settings.json` добавить провайдер. Не коммить этот файл и не помещай настоящий ключ в репозиторий.

```
{
  "env": {
    "BAILIAN_TOKEN_PLAN_API_KEY": "YOUR_TOKEN_PLAN_API_KEY"
  },
  "modelProviders": {
    "openai": [
      {
        "id": "qwen3.8-max-preview",
        "name": "[Token Plan Personal] qwen3.8-max-preview",
        "baseUrl": "https://token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1",
        "envKey": "BAILIAN_TOKEN_PLAN_API_KEY",
        "generationConfig": {
          "extra_body": {
            "enable_thinking": true
          }
        }
      }
    ]
  },
  "security": {
    "auth": {
      "selectedType": "openai"
    }
  }
}
```

```
}  
},  
"model": {  
  "name": "qwen3.8-max-preview"  
},  
"$version": 3  
}
```

Нужен именно dedicated API key со страницы Token Plan. Обычный pay-as-you-go ключ и адрес dashscope-intl с Token Plan не взаимозаменяемы.

Создание рабочей ветки

После baseline-коммита:

```
cd /Users/roman/Documents/GitHub/TRPG_charactersheet  
git status  
git switch -c qwen/social-library
```

git status перед запуском должен показывать только этот игнорируемый MD-файл как отсутствующий в статусе; tracked-код должен быть чистым.

Интерактивный безопасный запуск

```
qwen --model qwen3.8-max-preview
```

Внутри Qwen Code:

```
/permissions  
@QWEN_IMPLEMENTATION_README.md  
Выполни всё обязательное ТЗ по этапам. Сначала покажи краткий план и baseline commit, затем работай автономно. Не используй destructive git-команды и docker compose down -v. После каждого этапа запускай тесты и делай отдельный commit. Не останавливайся, пока критерии приёмки не выполнены или не найден реальный блокер, требующий решения пользователя.
```

Не включать --yolo: агент будет менять код, базу и выполнять Docker-команды, поэтому подтверждение потенциально опасных действий должно остаться включено.

Запуск с уже подставленным первым сообщением

```
qwen --model qwen3.8-max-preview --prompt-interactive "@QWEN_IMPLEMENTATION_README.md Выполни обязательное ТЗ по этапам, сохрани baseline и заверши только после полного docker compose run --rm check."
```

Если сессия стала слишком длинной, использовать /compress, затем снова указать незакрытые checklist-пункты. Для проверки модели использовать /model, для расхода - /stats, для продолжения прерванной сессии - /resume.

13. Источники конфигурации Qwen

- QwenCloud Qwen Code:

<https://docs.qwencloud.com/developer-guides/clients-and-developer-tools/qwen-code>

- QwenCloud Token Plan:

<https://docs.qwencloud.com/token-plan/overview>

- Qwen Code commands и @file:

<https://qwenlm.github.io/qwen-code-docs/en/users/features/commands/>

- Qwen Code sandbox и режимы разрешений:

<https://qwenlm.github.io/qwen-code-docs/en/users/features/sandbox/>

Готовность handoff

Baseline должен быть чистым, Qwen запускается в отдельной ветке, а итог принимается только после полного check и проверки сценария delete -> re-upload -> social interactions.